UGANDA CHRISTIAN UNIVERSITY

Distributed Data Processing Pipeline

Milestone Five: Technical Documentation and Report

Course Code:	DSC3219
Course Name:	Cloud and Distributed Computing
Programme:	Bachelor of Science in Data Science and Analytics
Examination Type:	Project-Based Examination

Prepared by:

Emmanuel Nsubuga
M23B38/005
B20811

Repository:
https://github.com/Cemputus/Distributed-Data-Processing-Pipeline

April 17, 2026

Contents

1 Executive Summary **5**

2 Introduction **6**
- 2.1 Project Background . . . 6
- 2.2 Project Objectives . . . 6
- 2.3 Project Scope . . . 6

3 Software Requirements Specification Summary **7**
- 3.1 System Intent and Architectural Direction . . . 7
- 3.2 Functional Requirements and Implementation Mapping . . . 7
- 3.3 Non-Functional Requirements in Practice . . . 8
- 3.4 Requirements Traceability to System Components . . . 8
- 3.5 Constraints and Engineering Trade-offs . . . 9
- 3.6 Summary . . . 9

4 System Overview **10**
- 4.1 High-Level Architecture . . . 10
- 4.2 Query and analytics paths . . . 11
- 4.3 Runtime Services . . . 11

5 Asset Inventory **12**
- 5.1 Software and Service Asset Inventory . . . 12
- 5.2 Runtime Resource Inventory . . . 13

6 Data Flow Diagram **14**

7 Sequence Diagram: Airflow to Spark ETL Flow **16**

8 Detailed Explanation of Implementation **17**
- 8.1 Frontend Application Layer . . . 17
  - 8.1.1 Dashboard and Navigation . . . 17
- 8.2 Dataset Upload . . . 17
- 8.3 Dataset Management and Ingestion . . . 18
- 8.4 ETL Orchestration and Distributed Processing . . . 19
- 8.5 Query and Analytics Paths . . . 20
- 8.6 Audit Logging and Traceability . . . 21
- 8.7 User Management and Role-Based Access Control . . . 22
- 8.8 Dashboard and Analytics . . . 22

9 Technical Justification of Software Choices and Settings **23**
9.1 Why Apache Airflow 23
9.2 Why Apache Spark 23
9.3 Why MinIO 24
9.4 Why PostgreSQL 24
9.5 Why Flask 24
9.6 Why React 24
9.7 Why Docker Compose 24
9.8 Configuration and Runtime Settings 24

10 Security and Operational Considerations **25**
10.1 Authentication and Session Handling 25
10.2 Authorization 25
10.3 Auditability 25
10.4 Service Isolation 25

11 Challenges Encountered and How They Were Overcome **26**

12 Learning Outcomes **27**

13 Conclusion **28**

List of Figures

1 High-level system architecture of the distributed data processing pipeline 10
2 Query and analytics paths architecture of the distributed data processing pipeline 11
3 Data Flow Diagram of the distributed data processing pipeline 15
4 Sequence Diagram for Airflow to Spark ETL execution flow 16
5 Dataset Upload and Processing Workflow 17
6 Dataset management interface showing uploaded datasets readiness classes and actions 18
7 ETL workspace showing Spark worker state and Airflow-linked ETL job records 19
8 Query and analytics architecture showing dashboard KPI path OSS warehouse CEN Query and optional BigQuery 20
9 Audit interface showing timestamped user actions outcomes and details 21
10 User management interface with roles and enable or disable controls 22
11 Dashboard and Analytics Interface 23

List of Tables

1 Functional Requirements to Implementation Mapping 8
2 Non-Functional Requirements and System Realisation 8
3 Requirements Traceability Matrix 9
6 Data Storage Structure and Purpose 14

1 Executive Summary

This report presents the technical documentation for the Distributed Data Processing Pipeline developed for the Cloud and Distributed Computing project. The system was designed and implemented as a practical demonstration of distributed storage parallel data processing workflow orchestration secure access control and analytical data delivery within a reproducible containerized environment.

The completed solution integrates Apache Airflow Apache Spark MinIO PostgreSQL Flask React and Docker Compose into one coordinated platform. The system ingests structured finance datasets through browser uploads and mounted data directories stores landed data in controlled storage zones registers ingestion metadata orchestrates ETL workflows through Airflow processes data in parallel using Spark and exposes curated outputs through a web-based operational console. The platform also supports audit logging role-based access control and optional cloud-based analytics through BigQuery while maintaining an open-source core stack.

A major achievement of the project is that it goes beyond a simple ETL script and delivers a complete distributed analytics platform. The system contains a functional React user interface for uploads dataset browsing ETL control query execution audit review and user management. The backend exposes API endpoints that coordinate processing storage and user operations. Spark workers provide distributed execution capacity while Airflow DAGs coordinate workflow dependencies. PostgreSQL acts as the open-source analytical result store and MinIO provides an S3-compatible storage layer for landed files and object-level data handling.

From an engineering perspective the project demonstrates practical implementation of key cloud and distributed computing concepts. These include distributed storage through MinIO and structured data zones parallel execution through Spark cluster workers orchestration through Airflow containerization through Docker Compose secure system access through authentication and role-based authorization and fault-aware operation through audit evidence workflow logs and recoverable services. The architecture was designed for reproducibility maintainability and demonstrability which makes it suitable for both academic evaluation and real-world extension.

The completed system therefore satisfies the project requirement of implementing an end-to-end distributed data processing pipeline with clear evidence of system design secure implementation orchestration processing analytical storage documentation and professional presentation readiness.

2 Introduction

2.1 Project Background

Modern organizations generate data from multiple operational points and often face difficulties in consolidating processing and analyzing such data in a scalable and secure manner. Traditional single-node systems are limited in throughput fault isolation and maintainability when data volume or workflow complexity increases. Distributed computing addresses this limitation by dividing data and tasks across multiple services and processing nodes.

This project was therefore designed as a distributed data processing pipeline that demonstrates how batch datasets can be ingested processed orchestrated secured and exposed for analytical use within a modular service architecture. The implementation focuses on a finance-themed dataset and uses an open-source stack that can run on a single developer machine while still reflecting real distributed system patterns.

2.2 Project Objectives

The major objectives of the project were:

- To design and implement a distributed data pipeline covering ingestion processing storage and analytics
- To demonstrate parallel data processing using Apache Spark
- To orchestrate ETL workflows using Apache Airflow
- To implement secure and role-aware access to operational services
- To store and expose analytics-ready outputs in a queryable analytical store
- To provide a user-facing application for uploads ETL monitoring querying audit review and administration
- To deliver a reproducible system using Docker-based deployment

2.3 Project Scope

The implemented project includes the following scope:

- CSV-based finance data ingestion through local datasets and browser uploads

- Landing storage and metadata tracking
- ETL orchestration through Airflow DAGs
- Distributed processing through Spark master-worker execution
- Curated analytics storage in PostgreSQL
- Query paths through SQLite-based CEN Query and optional BigQuery integration
- Authentication role-based access control session persistence and audit logging
- Web-based operational console implemented in React and Flask

3 Software Requirements Specification Summary

This section captures the core requirements that guided the design and implementation of the Distributed Data Processing Pipeline. The intention is not to restate the full SRS, but to show how the implemented system reflects the original design expectations in a practical and verifiable way.

3.1 System Intent and Architectural Direction

The system was designed as a containerised distributed data pipeline capable of handling batch ingestion, orchestrated execution, parallel processing, and analytical querying using an entirely open-source stack. The architecture separates ingestion, orchestration, processing, storage and user interaction into independent services coordinated through Docker Compose.

This separation ensures that each component operates within a clearly defined boundary, reducing coupling and allowing independent scaling and testing of services.

3.2 Functional Requirements and Implementation Mapping

The implementation aligns closely with the functional requirements defined during the specification phase. Instead of treating requirements as abstract statements, each one is realised through a concrete system component.

This mapping shows that each requirement category is directly reflected in the system structure, ensuring that the implementation remains grounded in the original specification.

Table 1: Functional Requirements to Implementation Mapping

Requirement Area	Description	Implementation Component
Data Ingestion	Upload and storage of batch datasets with traceability	Upload module, dataset registry, MinIO object storage
Workflow Orchestration	Scheduling and coordination of ETL workflows	Apache Airflow DAGs
Distributed Processing	Parallel data transformation and computation	Apache Spark standalone cluster (master + workers)
Analytical Storage	Storage of curated, queryable results	PostgreSQL analytics schema
User Interaction	System access, control and visibility	React frontend with Flask backend API
Ad-hoc Query	Read-only querying of processed datasets	CEN Query (SQLite-based execution)
Optional Cloud Analytics	Extended query capability (non-core)	Google BigQuery integration (optional)

3.3 Non-Functional Requirements in Practice

Beyond functionality, several system qualities influenced the final design.

Table 2: Non-Functional Requirements and System Realisation

Requirement	Implementation Approach
Reproducibility	Entire system deployed via Docker Compose with version-controlled configurations
Scalability	Spark workers and Airflow tasks operate independently, allowing horizontal scaling
Security	Role-based access control, token authentication, isolated container networking
Traceability	Dataset registry, Airflow logs and application-level audit logging
Portability	Open-source stack with no dependency on paid cloud infrastructure

These considerations ensured that the system is not only functional but also maintainable and suitable for controlled deployment.

3.4 Requirements Traceability to System Components

A clear traceability relationship exists between the defined requirements and the implemented architecture.

Table 3: Requirements Traceability Matrix

Requirement ID	Requirement Description	Mapped Component
FR-01 to FR-04	Data ingestion and storage	Upload service, MinIO, dataset registry
FR-05 to FR-08	Distributed processing	Spark jobs and execution cluster
FR-09 to FR-11	Analytical storage and validation	PostgreSQL analytics database
FR-12 to FR-14	Security and access control	Authentication system, RBAC, tokens
FR-21 to FR-23	Containerisation and orchestration	Docker Compose, Airflow
FR-24 to FR-28	Web application and query interface	React frontend, Flask API, CEN Query

This traceability confirms that the implementation remains consistent with the expectations defined during the requirements phase, without introducing unnecessary divergence.

3.5 Constraints and Engineering Trade-offs

The requirement to avoid paid infrastructure shaped the system significantly. Instead of relying on managed cloud services, the system uses Apache Airflow, Apache Spark, PostgreSQL and MinIO within a Docker-based environment.

The pipeline is intentionally batch-oriented. This allowed more focus on correctness, orchestration and data consistency rather than introducing the additional complexity of real-time streaming systems.

Security was implemented with a focused scope. Rather than over-engineering identity systems, the solution applies role-based access and controlled service communication, which is sufficient for the intended environment.

3.6 Summary

The final system reflects a strong alignment between specification and implementation. Each requirement category is represented by a concrete system component and the architecture maintains a balance between technical depth and practical deployability.

The result is a coherent distributed data pipeline that satisfies both the functional expectations and the operational constraints defined in the Software Requirements

Specification.

4 System Overview

4.1 High-Level Architecture

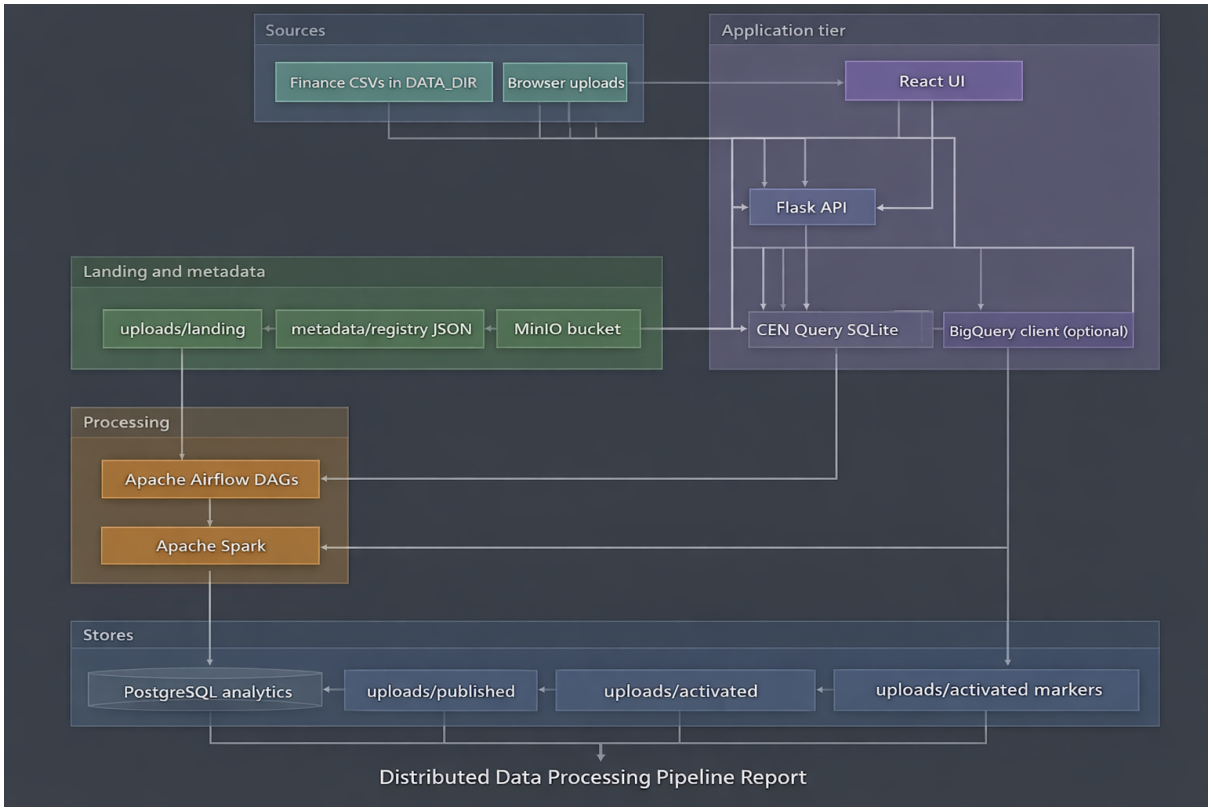


Figure 1: High-level system architecture of the distributed data processing pipeline

The architecture is organized into five logical layers. The first layer contains data sources in the form of finance CSV files and browser-based uploads. The second layer contains landing and metadata services including upload directories registry metadata and MinIO buckets. The third layer contains processing components namely Apache Airflow and Apache Spark. The fourth layer contains result stores including PostgreSQL analytics tables published outputs and activation markers. The fifth layer contains the application tier composed of the React interface Flask API CEN Query SQLite engine and optional BigQuery integration.

This layered arrangement improves separation of concerns and allows each responsibility to be isolated. Ingestion logic does not have to be embedded inside the query layer and processing logic is decoupled from storage and UI rendering. This makes the system easier to maintain test and defend technically.

4.2 Query and analytics paths

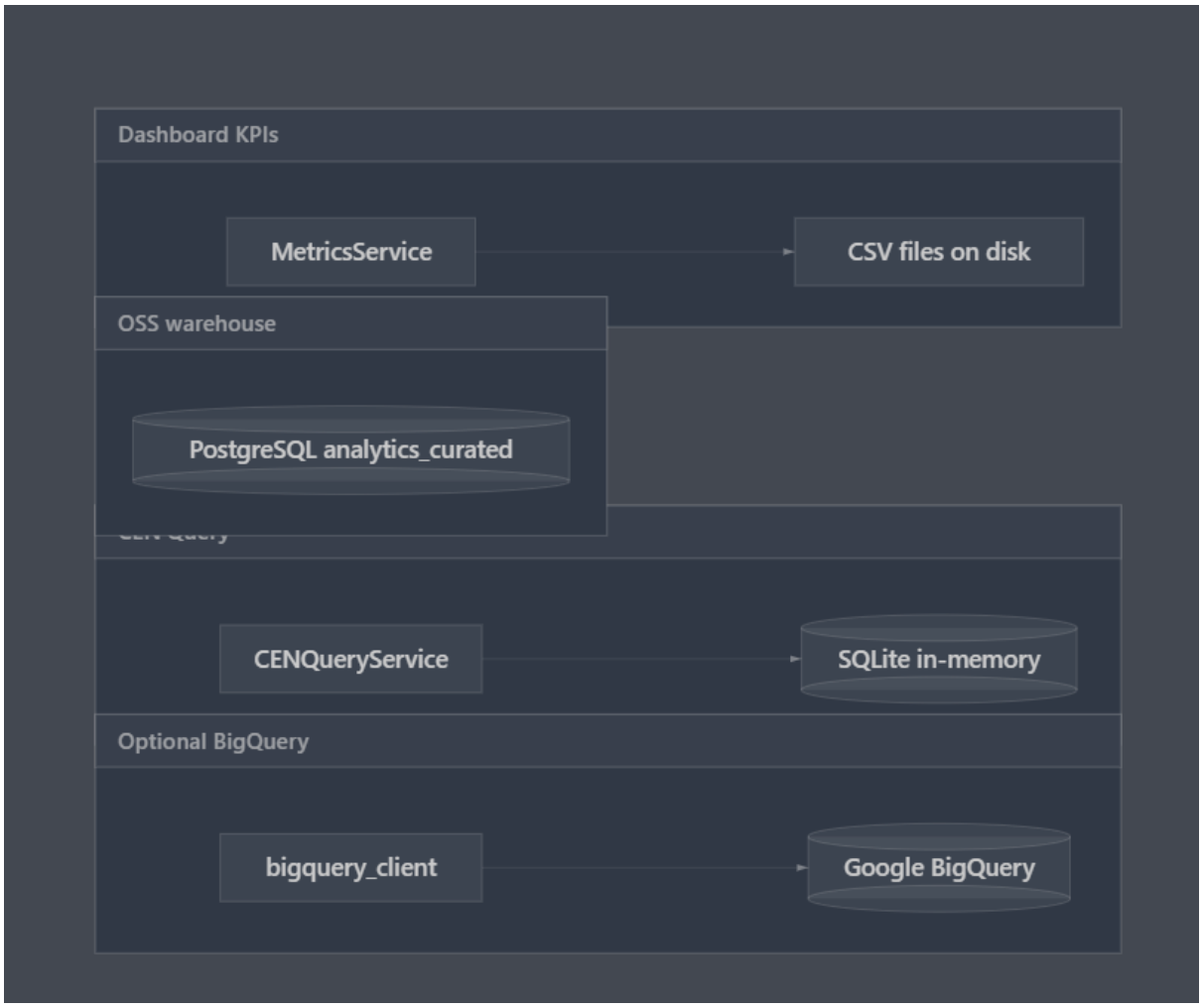


Figure 2: Query and analytics paths architecture of the distributed data processing pipeline

4.3 Runtime Services

The runtime stack was implemented using Docker Compose. Core services include:

- **Frontend:** React user interface for operational interaction
- **Backend:** Flask API for service coordination and business logic
- **PostgreSQL:** metadata and analytics result store
- **MinIO:** S3-compatible object storage for landing and file management
- **Airflow Webserver and Scheduler:** orchestration of ETL tasks
- **Spark Master and Worker:** distributed job execution environment

The system is therefore both logically distributed and operationally modular.

5 Asset Inventory

Although the milestone template refers to asset inventory in a general sense the implemented system is a software platform rather than a 3D modeling environment. For this project the asset inventory is therefore interpreted as the inventory of major system components services runtime resources data artifacts and their operational roles.

5.1 Software and Service Asset Inventory

Asset	Type	Description and Role
Apache Airflow	Orchestration engine	Coordinates ETL execution scheduling dependency handling task state and logging
Apache Spark Master	Processing coordinator	Accepts submitted jobs coordinates distributed execution across workers and exposes cluster status
Apache Spark Worker	Processing node	Executes Spark tasks using allocated CPU cores and memory
MinIO	Object storage	Stores landed files and provides S3-compatible storage semantics for distributed ingestion patterns
PostgreSQL analytics database	Analytical result store	Stores curated data and validation-ready warehouse-style tables
Flask API	Backend application service	Exposes operational endpoints for authentication uploads ETL triggers query execution and administration
React UI	Frontend application	Provides dashboard uploads datasets ETL query audit and users interface
SQLite in-memory engine	Query execution layer	Supports read-only ad hoc analytical querying through CEN Query

BigQuery client	Optional cloud integration	Provides optional access to Google BigQuery for read-only cloud analytics
Dataset registry JSON	Metadata asset	Tracks uploaded datasets classes readiness and related metadata
Audit logs JSON	Metadata asset	Stores auditable records of user actions and system events
Auth tokens JSON	Session asset	Supports persistence of issued authentication tokens across API restarts

5.2 Runtime Resource Inventory

Resource	**Observed Setting**	**Meaning**
Spark workers	1 worker	Demonstrates distributed execution capability in a constrained academic environment
Spark worker cores	2 cores	Available processing parallelism for tasks
Spark worker memory	1024 MB	Worker memory budget used for execution
Backend port	5000	API service endpoint
Frontend port	5173	Browser-accessible React interface
Spark Master UI port	8085	Monitoring access to Spark master state

Path or Artifact	Purpose
`data/fintech/*.csv`	Base finance datasets used for pipeline analytics and KPI generation
`uploads/landing/`	Initial landing zone for newly uploaded files
`uploads/processing/`	Intermediate processing workspace for data transformation
`uploads/published/`	Published outputs after transformation and promotion
`uploads/activated/`	Marker files used for analytics-ready gating
`uploads/metadata/datasets.json`	Dataset registry log and status tracking
`uploads/metadata/audit.json`	Audit logs capturing operation history
`uploads/metadata/auth.json`	Token persistence for session continuity

Table 6: Data Storage Structure and Purpose

6 Data Flow Diagram

Figure 3 presents the logical flow of data across the system. The diagram shows how finance CSVs and browser uploads enter the platform pass through controlled landing zones and metadata registration then move into orchestration and distributed processing before being stored and consumed analytically.

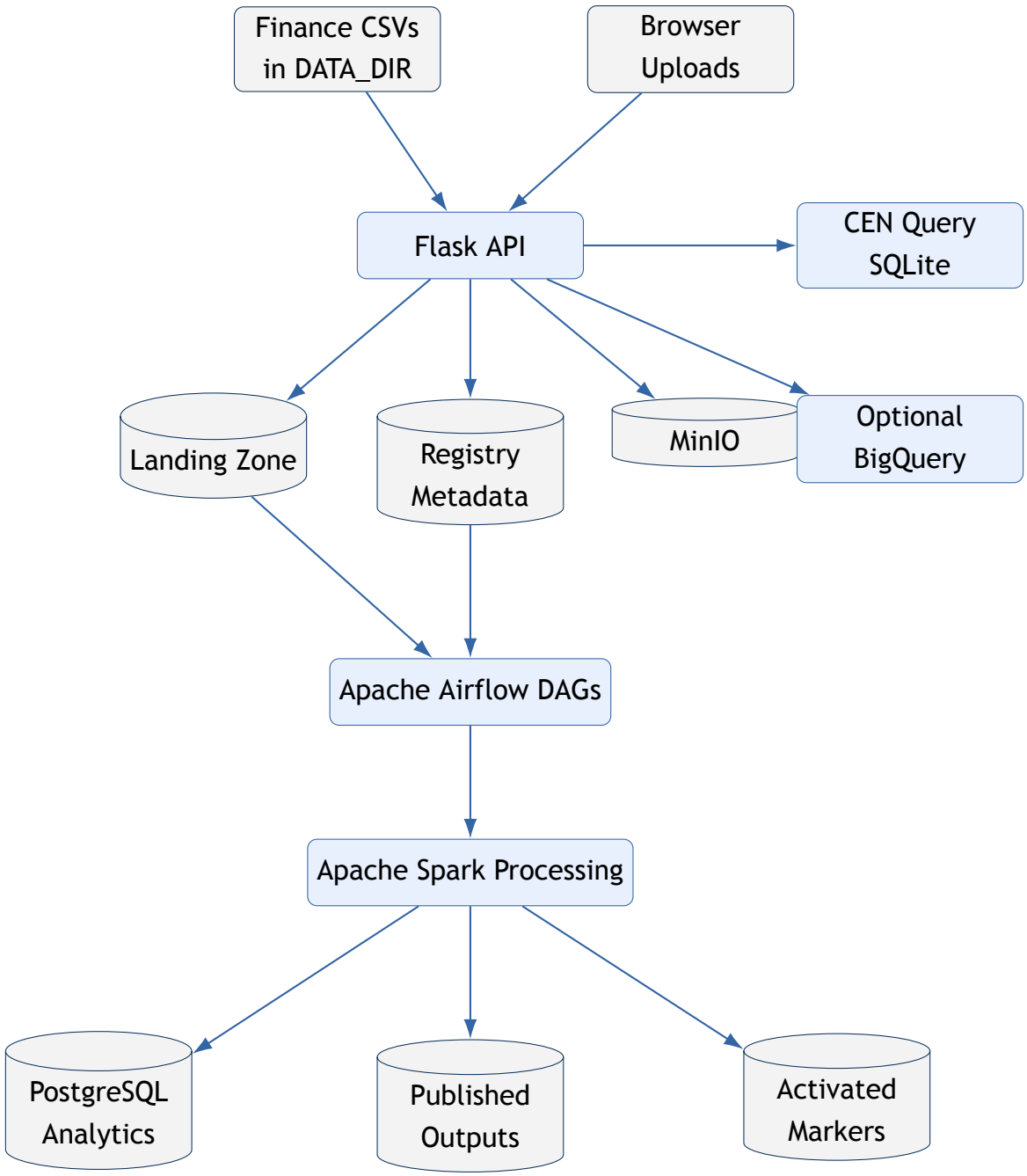


Figure 3: Data Flow Diagram of the distributed data processing pipeline

The data flow begins when users upload files or when the platform reads finance CSVs from the configured data directory. The Flask API coordinates intake and writes files into landing storage while also registering metadata. Airflow then uses the landing and metadata context to trigger pipeline execution. Spark performs distributed transformations and outputs curated results to PostgreSQL published outputs and activation markers. In parallel the API can expose CEN Query and optional BigQuery paths for query workloads.

7 Sequence Diagram: Airflow to Spark ETL Flow

The ETL control path is a central part of the project because it shows how orchestration and distributed execution interact. Figure 4 models the operational sequence from a user-triggered ETL request through Airflow scheduling Spark submission processing and result publication.

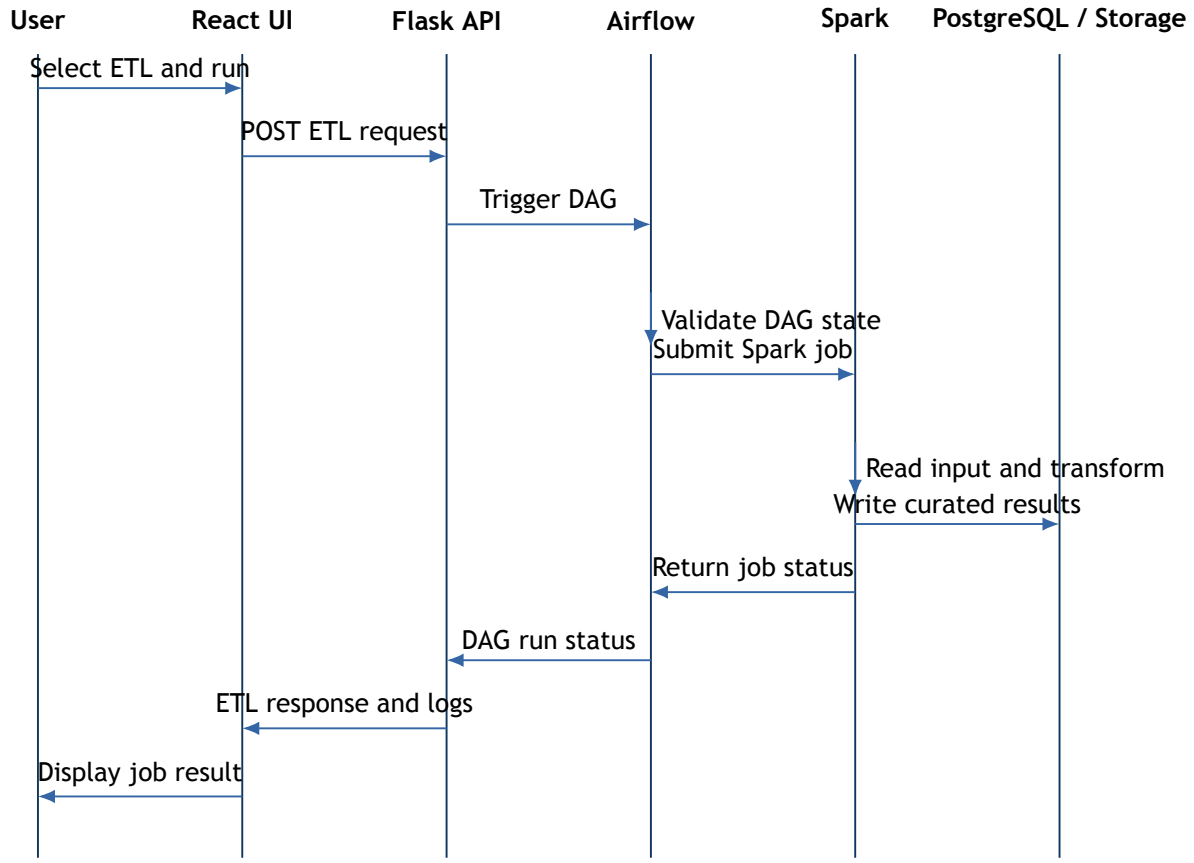


Figure 4: Sequence Diagram for Airflow to Spark ETL execution flow

The sequence begins when a user initiates ETL execution from the React interface. The frontend sends the request to the Flask API which forwards the workflow trigger to Airflow. Airflow checks DAG readiness and then delegates the computation to Spark. Spark reads landed data performs transformations and writes curated results to analytical storage. Status information propagates back from Spark to Airflow then to the backend and finally to the user interface where the result is displayed.

This sequence demonstrates the separation between workflow orchestration and computation. Airflow does not perform heavy analytics itself. Instead it manages dependencies scheduling retries and task state while Spark executes the actual distributed transformation workload.

8 Detailed Explanation of Implementation

8.1 Frontend Application Layer

The React frontend serves as the operational console of the platform. It provides a browser-based environment where users can authenticate navigate between modules upload datasets monitor ETL activity inspect audit information and manage users depending on assigned role permissions. This frontend improves usability and makes the distributed system more demonstrable during technical defense.

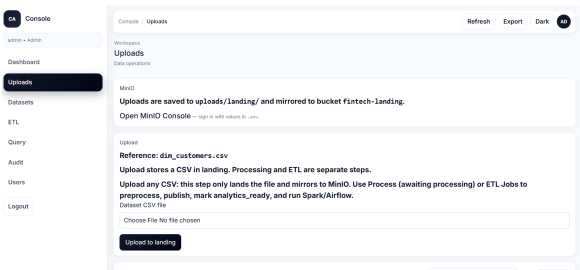
8.1.1 Dashboard and Navigation

The application layout includes a role-aware sidebar and top action bar. Users can move between dashboard uploads datasets ETL query audit and users pages. The interface uses stateful navigation and a consistent operational layout.

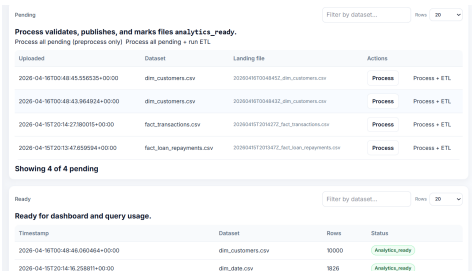
8.2 Dataset Upload

The dataset upload module provides the entry point for data ingestion into the pipeline. Users are able to upload CSV files through the web interface, after which the files are stored in the landing zone and mirrored into MinIO object storage. This separation ensures that ingestion is lightweight and independent from processing, allowing datasets to be validated and transformed in subsequent stages.

The interface clearly communicates that uploading only performs data landing while processing and ETL execution are handled separately. This design improves system robustness by decoupling ingestion from transformation and allows users to control when datasets are processed or published into analytics-ready states.



(a) Dataset upload interface and MinIO integration



(b) Processing queue showing pending and ready datasets

Figure 5: Dataset Upload and Processing Workflow

Uploaded datasets are tracked through a structured lifecycle. Files initially appear in

a pending state where preprocessing can be triggered. Once validated and processed, datasets are published and marked as `analytics_ready`, making them available for querying and dashboard visualization. This state-based tracking ensures consistency across ingestion, transformation, and analytics layers.

8.3 Dataset Management and Ingestion

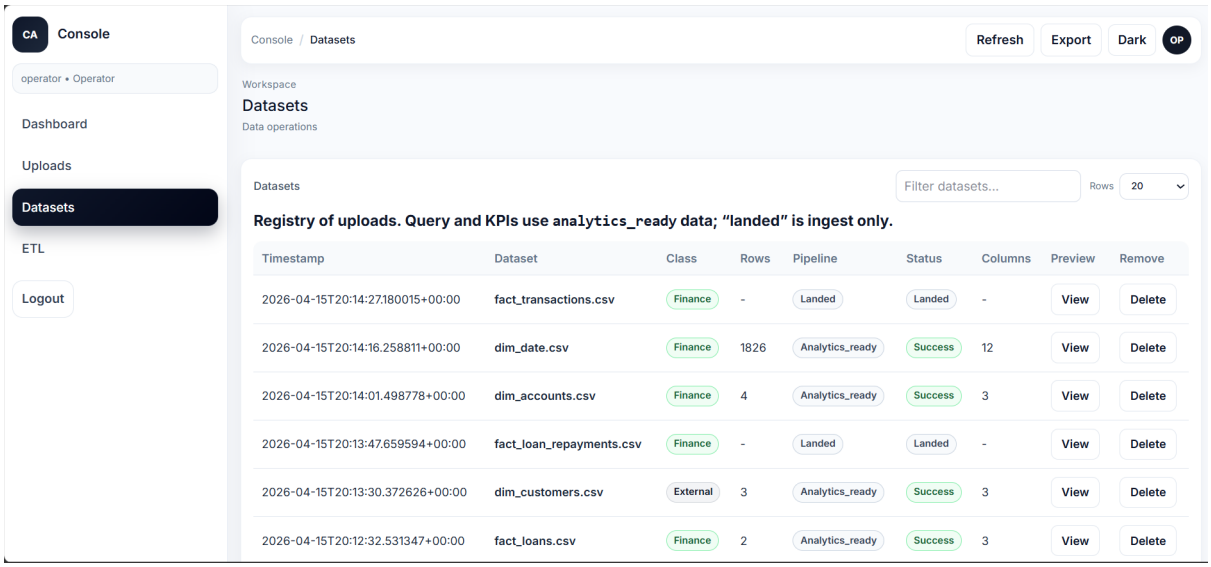


Figure 6: Dataset management interface showing uploaded datasets readiness classes and actions

The dataset module acts as the ingestion control surface. Uploaded files are registered with timestamp dataset name class row metadata pipeline state status and preview options. The interface distinguishes between *landed* data and *analytics_ready* data. This distinction is important because landed data represents ingest-only state while analytics-ready data has passed preparation or promotion requirements.

When a user uploads a file the backend stores the file into the relevant data zone and updates the metadata registry. This registry provides traceability and supports further operations such as ETL invocation filtering and validation.

8.4 ETL Orchestration and Distributed Processing

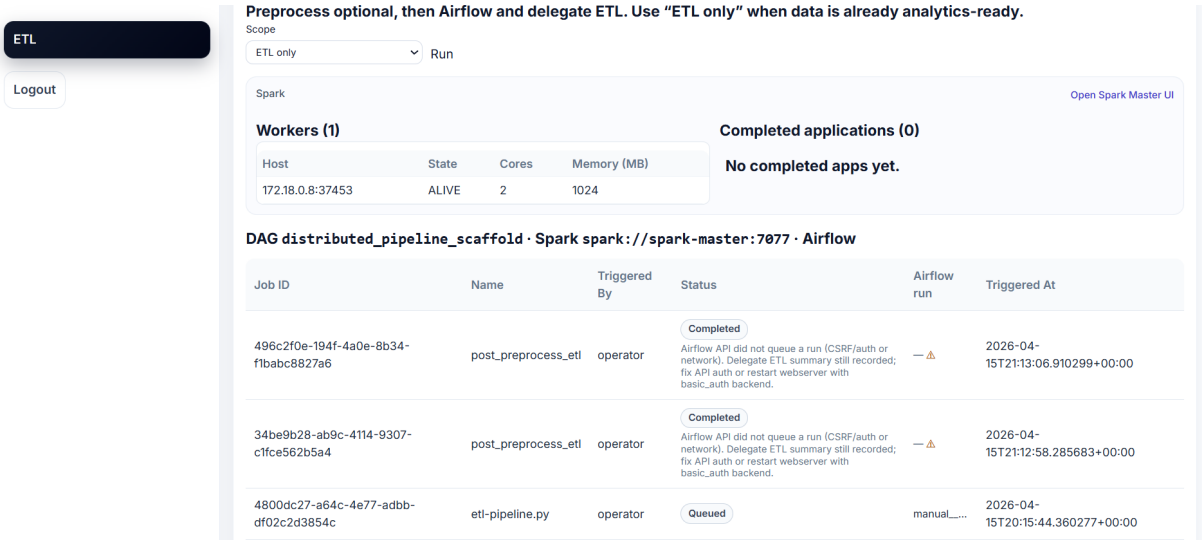


Figure 7: ETL workspace showing Spark worker state and Airflow-linked ETL job records

The ETL module exposes the workflow execution path. Users can choose ETL scope and run processing tasks. The interface also shows Spark cluster state including active worker count host state available cores and memory. Job records include identifiers job names triggering user Airflow linkage status and execution time.

Apache Airflow was used as the orchestration engine because it enables formal workflow definition through DAGs. This approach is superior to manually chained scripts because it introduces task dependencies monitoring retry support and run-level traceability. In the implemented system Airflow DAGs coordinate the movement from dataset intake to Spark execution and result publication.

Apache Spark was selected as the distributed processing engine because it can partition data and execute transformation tasks across worker resources. Even though the academic deployment used a modest worker size the architecture remains scalable because additional workers can be added within the same cluster model. Spark provides the computation layer for transformations joins aggregations and ETL operations required by the pipeline.

8.5 Query and Analytics Paths

Figure 8: Query and analytics architecture showing dashboard KPI path OSS warehouse CEN Query and optional BigQuery

The system supports multiple analytical access paths. Dashboard KPIs are derived from finance CSV data through a metrics service. A separate CEN Query service loads analytics-ready datasets into an in-memory SQLite engine for read-only SQL queries. This provides a lightweight but effective analytical query path for interactive use. In addition the system can optionally connect to Google BigQuery through a backend client for extended cloud analytics scenarios.

PostgreSQL acts as the open-source warehouse-style analytical store. Curated outputs generated by ETL processes are persisted in PostgreSQL to support validation queries and structured result inspection. This aligns well with the project requirement that the system must produce queryable analytical outputs.

8.6 Audit Logging and Traceability

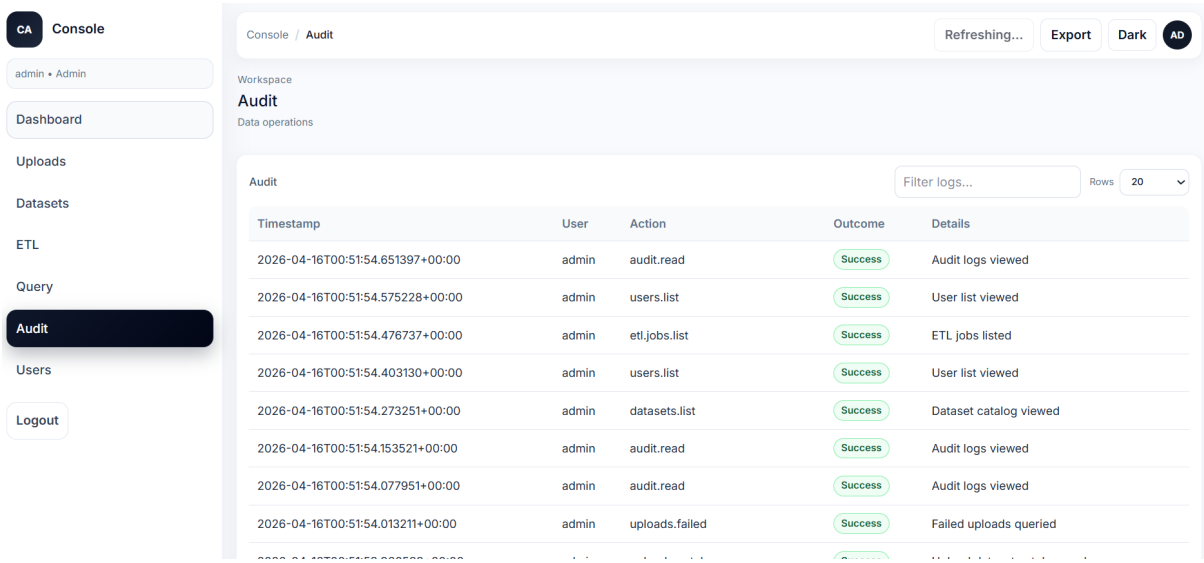


Figure 9: Audit interface showing timestamped user actions outcomes and details

Audit logging is one of the strongest operational features of the implemented system. The audit module records timestamp user identity action outcome and details for system operations. This includes dataset listing ETL job access user management actions and audit viewing itself.

This capability improves accountability and makes the system easier to defend from a security and operations perspective. It also provides practical evidence that the platform captures meaningful operational events rather than functioning as an opaque black box.

8.7 User Management and Role-Based Access Control

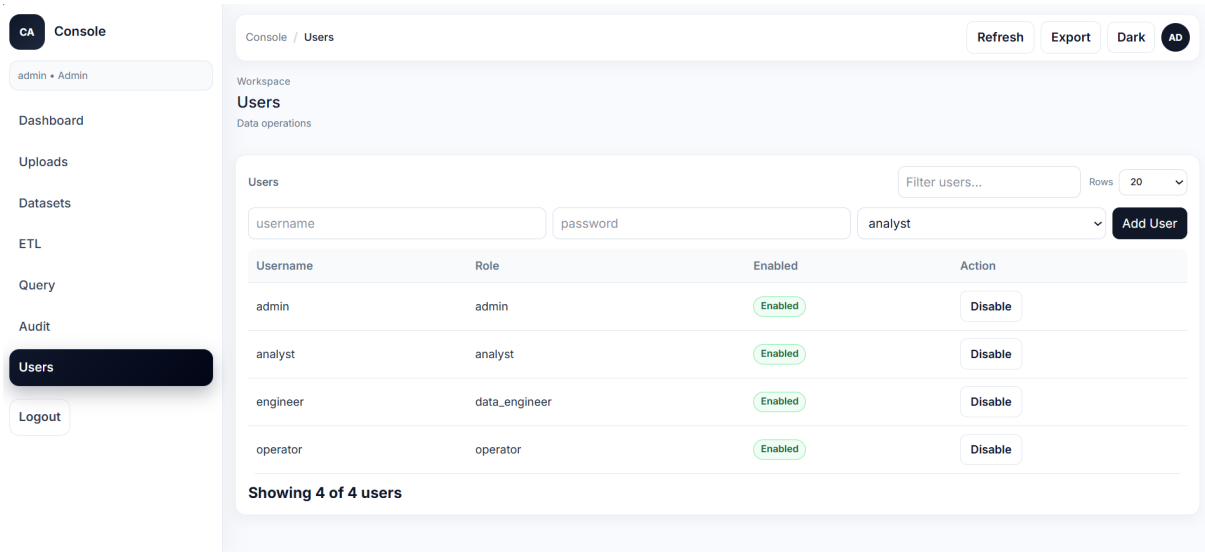


Figure 10: User management interface with roles and enable or disable controls

The user management module allows administrative control over the identities allowed to operate within the system. Roles include `admin` `analyst` `data_engineer` and `operator`. Each role is tied to a specific access pattern in the backend and reflected in the UI.

The existence of role-based access control is technically important because it demonstrates that the system does not treat all users equally. Instead it enforces controlled access to operational capabilities such as audit viewing user management ETL execution and analytical interaction. This satisfies a major part of the security expectations for the project.

8.8 Dashboard and Analytics

The dashboard and analytics module provides a consolidated view of processed data and system insights. It presents key metrics such as customer counts account volumes transaction activity loan statistics and fraud alerts in a structured and intuitive layout.

The dashboard integrates directly with the analytics layer backed by PostgreSQL, enabling fast query execution and real-time data visualization. Aggregated metrics are displayed alongside visual summaries such as top-performing merchants transaction distributions and financial activity patterns.

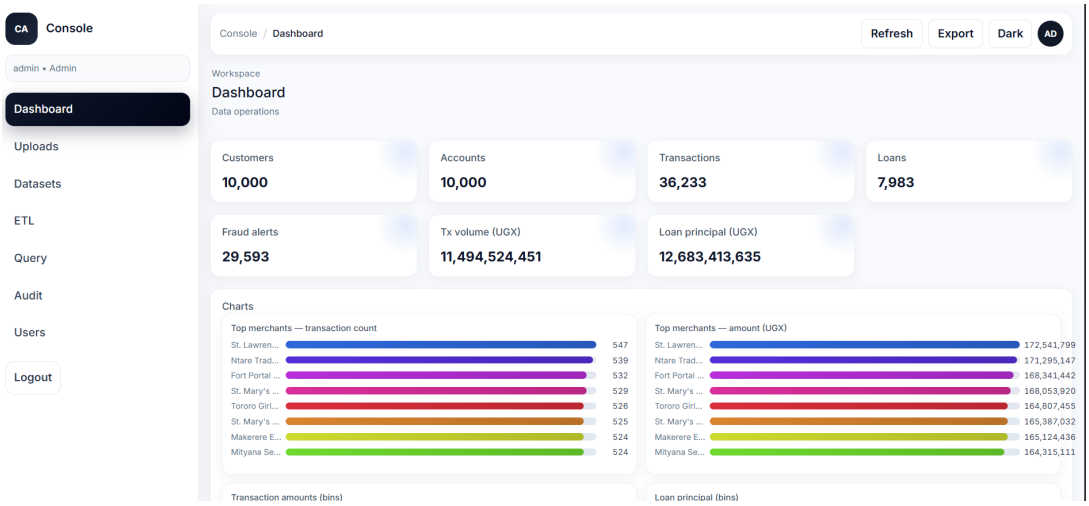


Figure 11: Dashboard and Analytics Interface

This component demonstrates the final stage of the data pipeline where raw and processed data are transformed into actionable insights. By connecting ingestion processing and storage to a user-friendly interface, the system enables effective monitoring analysis and decision-making based on curated datasets.

9 Technical Justification of Software Choices and Settings

9.1 Why Apache Airflow

Apache Airflow was selected as the workflow orchestration engine because the project required a coordinated multi-stage pipeline rather than isolated scripts. Airflow uses DAG-based workflow definitions which provide explicit task ordering monitoring scheduling reruns and retry mechanisms. These features are valuable in distributed systems where visibility and execution control matter. Airflow therefore served as the control plane of the ETL architecture.

9.2 Why Apache Spark

Apache Spark was chosen because it is an industry-standard distributed processing engine suitable for batch analytics transformations joins and aggregations. Spark supports data partitioning and worker-based execution which aligns with the project requirement to demonstrate parallel and distributed processing. Its integration with Airflow and containerized deployment also made it practical for the chosen stack.

9.3 Why MinIO

MinIO was selected as the landing storage layer because it provides an S3-compatible object storage interface within an open-source lightweight deployment model. This makes it suitable for academic use while still reflecting cloud-native storage patterns. MinIO therefore enabled object-based storage without introducing paid infrastructure dependency.

9.4 Why PostgreSQL

PostgreSQL was used as the open-source analytical result store because it is stable mature widely documented and easy to validate through SQL. It provides a strong relational foundation for curated outputs and was appropriate for storing analytics-ready results in a warehouse-style format.

9.5 Why Flask

Flask was chosen as the backend framework because it is lightweight modular and suitable for building API-driven services. The project did not require a heavy backend framework. Flask made it easier to implement authentication upload handling ETL endpoints query services and administration features in a clean and extensible way.

9.6 Why React

React was chosen as the frontend framework because it supports component-based UI development and interactive state-driven interfaces. Since the project required multiple operational pages and a responsive admin-style console React was a suitable choice for building a maintainable user interface.

9.7 Why Docker Compose

Docker Compose was used because the project contains multiple tightly related services that must run together in a reproducible manner. Compose allows each component to be isolated in its own container while still participating in a shared application network. This improves portability repeatability and ease of demonstration.

9.8 Configuration and Runtime Settings

Important runtime settings observed in the implementation include:

- Spark worker availability with 2 cores and 1024MB memory
- Backend service running on port 5000
- Frontend service running on port 5173
- Spark Master UI exposed for cluster visibility
- Airflow used to control DAG execution and ETL status
- Persistent metadata files used for audit logs dataset registry and auth token continuity

These settings were sufficient for an academic deployment while still enabling meaningful distributed processing behavior.

10 Security and Operational Considerations

10.1 Authentication and Session Handling

The platform uses authentication tokens to protect API access. Session persistence is supported through metadata-backed token storage which means user sessions can survive API restarts under controlled conditions. This improves user continuity and demonstrates practical handling of session state in a distributed application.

10.2 Authorization

Authorization is implemented through backend-enforced roles. Users are not granted unrestricted access to all modules. This ensures that only appropriate users can perform administrative or operational tasks such as ETL execution user management and certain forms of data access.

10.3 Auditability

Audit logs serve as a security and governance feature. Every important action can be tied to a user timestamp action type and outcome. This provides operational evidence and supports accountability.

10.4 Service Isolation

Because the system is containerized each major service runs in its own containerized environment. This improves modularity and reduces direct interference between

unrelated components. It also supports safer debugging and clearer responsibility boundaries.

11 Challenges Encountered and How They Were Overcome

A major implementation challenge arose from the need to coordinate several containerized services as one coherent platform. The React frontend Flask backend PostgreSQL database Apache Airflow services MinIO object storage and Apache Spark cluster each had to be configured with the correct network settings service names port mappings mounted volumes and environment variables. Because these services were interdependent even a small configuration mismatch could interrupt communication and prevent the pipeline from running correctly. This challenge was addressed by structuring the environment with Docker Compose defining service relationships clearly and validating communication incrementally until the upload workflows API endpoints orchestration layer storage layer and processing layer operated reliably as one system.

Another significant challenge emerged in the integration between Apache Airflow and Apache Spark. Workflow orchestration and distributed computation are related but distinct responsibilities and this meant the system had to ensure that Airflow could control execution without taking over the computational role that belonged to Spark. The handoff between orchestration and processing therefore required careful design so that DAGs triggered jobs correctly job states remained visible and ETL activity could still be monitored from the application interface. This was resolved through deliberate DAG design runtime verification of trigger behavior and continuous inspection of ETL job records so that orchestration remained observable while computation remained delegated to Spark.

Maintaining consistency across landed datasets analytics-ready datasets and metadata records also proved to be a technical challenge. Since datasets could move through multiple operational states the system needed a dependable way to distinguish raw uploaded files from prepared analytical assets. Without that distinction there was a risk that dashboards queries or ETL routines might consume incomplete data or data that had not yet been validated for analytics. The issue was addressed by introducing explicit registry files clearly separated operational data zones and state-aware labeling within the interface so that every dataset could be tracked according to its current readiness level.

Security and traceability introduced another layer of complexity. The platform needed meaningful access control and audit capability yet it also had to remain manageable within the scope of an academic project. A security model that was too weak would reduce the credibility of the implementation while one that was too complicated would make the system difficult to complete and maintain. This challenge was addressed by adopting a focused security design built around clearly defined user roles persistent authentication tokens and structured audit logging. That approach provided practical control over access system accountability and continuity of user sessions without introducing unnecessary architectural complexity.

Preserving the open-source reproducible nature of the project while still demonstrating modern cloud and distributed computing concepts was also an important concern. Many production data systems depend on managed commercial infrastructure but such a direction would have reduced portability and made the project less suitable for academic evaluation. This challenge was addressed by building the core platform around Docker Compose Apache Airflow Apache Spark MinIO PostgreSQL Flask and React. As a result the final system remained reproducible on local infrastructure technically defensible in an academic setting and aligned with the requirement for a self-contained distributed implementation.

12 Learning Outcomes

This project contributed significantly to understanding and applying cloud and distributed computing concepts. The major learning outcomes include:

- Understanding how distributed pipelines are decomposed into ingestion processing storage and consumption layers
- Learning how workflow orchestration differs from execution and why tools like Airflow are needed
- Gaining practical experience with Spark-based distributed processing
- Understanding the importance of object storage metadata management and curated analytical outputs
- Learning how authentication authorization and auditability strengthen system credibility
- Developing experience with Docker-based multi-service deployment
- Improving ability to document and justify architecture in a professional tech-

nical format

13 Conclusion

This project successfully delivered a distributed data processing pipeline that integrates ingestion orchestration distributed processing analytical storage and user-facing interaction into one coherent platform. The completed system demonstrates the practical application of Apache Airflow Apache Spark MinIO PostgreSQL Flask React and Docker Compose in building a modular analytics architecture.

The final implementation is not limited to a simple ETL demonstration. It includes a full operational console dataset management ETL control query services audit evidence and user administration. This makes the project technically stronger and more aligned with real-world distributed data platforms.

From an academic perspective the project demonstrates understanding of distributed storage workflow orchestration parallel processing secure access controlled operations and technical documentation. From an engineering perspective it provides a strong foundation that can be extended in future to include more workers larger datasets stronger warehouse modeling streaming support and more advanced monitoring.

The project therefore fulfills the course objective of implementing and defending a distributed data processing pipeline with clear technical depth practical execution and professional reporting quality.

References

[1] Apache Software Foundation, "Apache Spark Documentation." [Online]. Available: https://spark.apache.org/docs/latest/. [Accessed: Apr. 16, 2026].

[2] Apache Software Foundation, "Apache Airflow Documentation." [Online]. Available: https://airflow.apache.org/docs/. [Accessed: Apr. 16, 2026].

[3] Apache Software Foundation, "Apache Hadoop Documentation." [Online]. Available: https://hadoop.apache.org/docs/. [Accessed: Apr. 16, 2026].

[4] Docker Inc., "Docker Documentation." [Online]. Available: https://docs.docker.com/. [Accessed: Apr. 16, 2026].

[5] MinIO Inc., "MinIO Documentation." [Online]. Available: https://min.io/docs/. [Accessed: Apr. 16, 2026].

[6] The PostgreSQL Global Development Group, "PostgreSQL Documentation." [Online]. Available: https://www.postgresql.org/docs/. [Accessed: Apr. 16, 2026].

[7] Pallets, "Flask Documentation." [Online]. Available: https://flask.palletsprojects.com/. [Accessed: Apr. 16, 2026].

[8] Meta Open Source, "React Documentation." [Online]. Available: https://react.dev/. [Accessed: Apr. 16, 2026].

[9] Google Cloud, "BigQuery Documentation." [Online]. Available: https://cloud.google.com/bigquery/docs. [Accessed: Apr. 16, 2026].

[10] Project, "MILESTONE1_SRS.md," Apr. 2026.

[11] Project, "ARCHITECTURE.md," Apr. 2026.